Treasure Row

Donovan Griffin, Chase Jones, Noah Higgs, Jonah Goodwine,
Cameron Ridgley, Ricki Davis, & Chi Martin



Functional Specifications

Table of Contents

1.0 Introduction 5
1.3 Software context 5
1.4 Major constraints 6
2.0 Usage scenario 6
2.1 User profiles 7
2.2 Use-cases 7
2.3 Special usage considerations 8
3.0 Data Model and Description 8
3.1 Data Description 8
3.1.1 Data Objects 9
3.1.1.1 User 9
3.1.1.2 Storefront 9
3.1.1.3 Listing 10
3.1.1.4 Listing Image 11
3.1.1.5 Listing Size 11
3.1.1.6 Wishlist 11
3.1.1.7 Order 12
3.1.2 Relationships 13
3.1.3 Complete data model 14
3.1.4 Data dictionary 14
4.0 Functional Model and Description 17
4.1 Description for Function n 17
Registration and Login 17
Storefronts 18
Browse & Search 18
Wishlist 19

Cart & Checkout 19
Treasure Tokens 20
4.1.1 Processing narrative (PSPEC) for function n 22
4.1.2 Function n flow diagram 24
4.1.3 Function n interface description 24
4.1.4 Function n transforms 25
4.1.4.1 Transform 1 description (processing narrative, PSPEC) 25
4.1.4.2 Transform 1 interface description 26
4.1.4.3 Transform 1 lower level flow diagrams 26
4.1.4.4 Transform 2 description 26
4.1.4.5 Transform 2 interface description 27
4.1.4.6 Transform 2 lower-level flow diagrams 27
4.1.4.7 Transform 3 description 27
4.1.4.9 Transform 3 lower-level flow diagrams 28
4.1.5 Performance Issues 28
4.1.6 Design Constraints 29
4.2 Software Interface Description 29
4.2.1 External machine interfaces 29
4.2.2 External system interfaces 29
4.2.3 Human interface 29
4.3 Control flow description 30
5.0 Behavioral Model and Description 46
5.1 Description for software behavior 46
5.1.1 Events 46
5.1.2 States 47
5.2 State Transition Diagrams 47
5.3 Control specification (CSPEC) 47

6.0 Restrictions, Limitations, and Constraints 48
7.0 Validation Criteria 49
7.1 Classes of tests 49
7.2 Expected software response 50
7.3 Performance bounds 50
8.0 Revision Logs 50

1.0 Introduction

1.1 Goals and objectives

Treasure Row is a browser-based campus marketplace for Hampton University students. Its documented functionality includes account authentication, storefront and listing management, browsing, wishlists, persistent carts, checkout using simulated Treasure Tokens, order tracking, escrow and pickup confirmation, return-request tracking, and administrative controls. Features should be identified as implemented, partially implemented, or planned so the scope accurately reflects the current project state.

1.2 Statement of scope

The software developed for this cloud-based marketplace will allow Hampton University students to be able to buy and sell original clothing. The system will include user authentication, listing management, and database-backed storage through a web-based user interface.

- **Inputs:**

    - User login credentials
    - User-created listing information (title, description, photos, category, etc.)
    - User actions (deleting listings, filters for listings, edit/add listings, etc.)
- **Processing functionality:**
    - Authentications and authorization of user using secure password hashing
    - Creation, storage, retrieval, and updates of listings and purchases
    - Listing ownership and authorization, ensuring users to edit only the listing they created
    - Cloud-hosted database storage using AWS
- **Outputs:**
    - A catalog of active marketplace listings
    - User-owned listings with options to edit or delete

1.3 Software context

For any and all software being used for our product, the following behavioral requirements are needed:

- Security
    - The users login and listing information must be secure at all times
- Scalability
    - The marketplace must be able to store a vast amount of users and user listings while the number for both is continuously growing
- Reliability
    - The marketplace must have little to no crashes and cannot to vulnerable to cyber threats
- Usability
    - Users must be able to access the marketplace across macOS, Windows, and multiple web browsers

1.4 Major constraints

The following constraints will affect product development:

- **Technical Constraints:**
    - **Cross-platform compatibility:** The system must function consistently on macOS, Windows, and across web-browsers
    - **Listing ownership:** The system must be developed using AWS which requires compatibility with AWS hosting, networking, and database solutions.
    - **AWS Free tier:** Since the team only has access to AWS Free Tier, the system must be deployed using the free tier, limiting cloud resources and requiring efficient configuration
- **Resource Constraints:**
    - **Time Constraints:** The product has a set timeline that it must be complete within. If it is not completed within the given timeline, it will need to be handed off to the next class to be refined.

2.0 Usage scenario

Users of *Treasure Row* will consist of Hampton University students who will be able to access the platform through a browser and application. All users will gain access to the application by registering for an account using verified university credentials and a secure username and password. Once authenticated, users will be directed to our homepage, where they will be able to browse the catalog for available clothing items from verified student brands. Shoppers will also be able to view details for each item as well as filter through the items. Student brand owners will be able to create an account for their businesses and create new listings by entering all the information on the clothing. Users are able to edit and delete any listing that they created, while administrators have permissions to manage all listings. After a user has successfully finished any task of their choosing, they can log off of the system securely.

2.1 User profiles

Treasure Row is made to support multiple user categories. Each category consists of different roles, permissions, and goals.

Student Shopper
The student shopper is a Hampton University student who will use the platform to browse the clothing listings given by the student brand owner. The goal of this user is to discover and view any available items posted by the brand owners. Shoppers can view listings, search and filter items, and message sellers, but they cannot modify listings.

Student Brand Owner
A student brand owner is a Hampton University student who manages and owns a clothing brand and contributes their brand to *Treasure Row*. The user is able to create, edit, delete, and add to the browser listing. Brand owners are responsible for maintaining the information for their products and managing their listings.

Administrator
An administrator is a user given privileges and responsibilities to oversee the system. Administrators are given access to all listings and can modify them (i.e., edit, remove, etc.) if any listings violate platform guidelines. This role ensures the system's integrity, policy is enforced, and overall platform management.

2.2 Use-cases

The following use-cases represent the primary interactions users have with Treasure Row:

- User registers for an account using Hampton University credentials or similar
- User logs into the system using a secure username and password
- Student shopper browses and searches marketplace listings
- Student brand owner creates a new clothing listing
- Student brand owner edits an existing listing they own
- The student brand owner deletes a listing they own
- Administrator views, edits, or deletes any listing
- User logs out of the system securely

2.3 Special usage considerations

The following special use cases explain exceptional or restricted scenarios within Treasure Row's programming that ensure security, integrity of the platform, and enforcement of policy, procedures, and rules.

The system confirms listing ownership in the event of an unauthorized listing edit, which occurs when a student user tries to edit a listing they do not own. Access is refused and a suitable error message is shown if the user is not the owner. The listing is still in effect.

Administrators can change, remove, or edit any listings if they are found to violate any platform's rules. This ensures system compliance and preserves integrity by updating the platform's marketplace to reflect the accurate representation of the platform.

Lastly, authentication fails and an error message appears if a user enters incorrect login information. The user must try to log in again using legitimate credentials since they are still logged out.

3.0 Data Model and Description

This section describes the information domain for Treasure Row. There will be descriptions identifying the data objects the system manages/affects and the relationships among them. Additionally, the data model included, supports the core requirements for the system such as such as user authentication, storefront creation, listing management, ownership enforcement, inventory tracking, wishlist management, listing storage, checkout, order tracking, return handling, and administrative control.

All structured data is stored in a cloud-hosted relational database using AWS RDS (PostgreSQL)

3.1 Data Description

Treasure Row manages several categories of data:

- User account data
- Storefront and listing data
- Order and transaction data
- Cart data
- Return request data
- Wishlist data

More specifically, user account data supports secure authentication and role-based authorization.

Storefront and listing data supports product creation, modification, and ownership enforcement within the marketplace. Order and cart data supports the checkout lifecycle and transaction tracking. Wishlist data supports user engagement by allowing shoppers to save items of interest. All data elements are stored within a structured relational database to maintain consistency and enforce relationships between entities.

3.1.1 Data Objects

The following data objects represent the primary entities required to support the system's functional requirements.

3.1.1.1 User

The User object represents a registered Hampton University student or administrator who accesses Treasure Row. Each user is assigned a role that determines their level of access and permissions within the system.

Users operate under a unified account model, meaning a signal account may function as both a shopper and a brand owner depending on whether a storefront has been created.

Attributes:

- user_id (unique identifier)
- username
- email
- password_hash (stored securely using hashing)
- role (shopper, brand_owner, admin)
- Created_at (timestamp of account creation)

3.1.1.2 Storefront

The Storefront object represents a brand profile created by a user. When a user creates a storefront, they gain seller capabilities within the system.

A storefront serves as the organizational entity under which listings are created and managed.

Attributes:

- storefront_id
- owner_id (references User)
- brand_name
- bio
- logo_url
- banner_url
- contact_info
- preview_image_1
- preview_image_2
- preview_image_3
- preview_image_4
- categories
- is_active
- created_at
- updated_at
-

3.1.1.3 Listing

The Listings object represents an item offered for sale by a brand owner. Each listing is associated with a specific storefront.

Listings may operate under two fulfillment models:

- **Inventory-based:** Limited quantity available
- **Made-to-order: (preorder):** Produced after purchase with no fixed inventory count

The system enforces inventory validation for inventory-based listings and automatically updates status when quantities reach zero.

Attributes:

- listing_id (unique identifier)
- storefront_id (references Storefront)
- title
- description
- price
- fulfillment_type
- quantity_on_hand
- sizes_available
- is_made_to_order
- status
- storefront_restore_status
- deleted_restore_status
- created_at
- updated_at

Inventory control is a significant factor and for inventory based listings, quantity_avilable is displayed to shoppers when browsing or viewing a listing (e.g., sizes/quantity available). The system validates the quantity available when an item is added to cart and again at checkout.

For made-to-order listings, quantity limits do not apply, and orders can be placed without stock manipulation

3.1.1.4 Listing Image

The Listing image object represents image records associated with a listing.

Images are stored externally in Amazon S3, while the corresponding image URLs are stored in the database. A listing may contain multiple images, with one designated as the primary image.

Attributes:

- Image_id
- Listing_id (references Listing)
- Image_url
- Is_primary
- Created_at

3.1.1.5 Listing Size

The Listing Size object supports size-based inventory tracking for listings that require multiple size options.

Each size entry maintains a quantity value and is associated with a specific listing.

Attributes:

- Size_id
- Listing_id (references Listing)
- Size_label
- Quantity
- Created_at

3.1.1.6 Wishlist

The Wishlist object represents saved listings associated with a user account. It allows shoppers to store items for future review or purchase.

Attributes:

- Wishlist_id
- User_id (references user)
- Listing_id (references Listing)
- created_at

3.1.1.7 Order

The Order object represents a transaction initiated by a shopper during checkout. It tracks the lifecycle of a purchase from initiation through fulfillment.

Each order records both the buyer and the seller to support transaction history tracking and administrative oversight.

Attributes:

- order_id (unique identifier)
- user_id (references User)
- confirmation_number (unique, system-generated)
- buyer_first_name
- buyer_last_name
- buyer_email
- buyer_contact
- buyer_meeting_location
- total_amount
- Order status values: SUBMITTED, COMPLETED, and CANCELLED
    - SUBMITTED means checkout succeeded and at least one associated escrow has not reached a terminal outcome
    - COMPLETED means all required seller escrows have been released after pickup confirmation
    - CANCELLED means all applicable escrows have been refunded
    - Define escrow status separately, including the states for held, released, refunded, and locked for administrative review, and document the permitted transitions between states.created_at

3.1.1.8 Order Item

The Order Item object represents an individual line item within an order. Each order may contain multiple order items corresponding to the listings purchased during checkout.

Attributes:

- order_item_id (unique identifier)
- order_id (references Order)
- listing_id (references Listing)
- item_name
- quantity
- price_at_purchase
- selected_size

3.1.1.9 Cart Item

The Cart Item object represents a listing added to a user's shopping bag. Cart items are stored persistently in the database and loaded into the session on login. The cart is cleared upon successful checkout.

Attributes:

- cart_item_id (unique identifier)
- user_id (references User)
- item_id (references Listing)
- item_name
- price
- quantity
- size
- created_at

3.1.1.10 Return

The Return object represents a return request submitted by a user after a purchase. It tracks the reason, damage status, and optional damage image uploaded to S3.

Attributes:

- return_id (unique identifier)
- user_id (references User)
- order_number
- reason
- has_damage
- damage_image_url (S3 URL, optional)
- status (pending, reviewed, resolved)
- created_at
- updated_at
- is_deleted

3.1.1.11 Message (Jonah Goodwine)

The Message object represents one text message sent from one user to another, usually about a specific listing. A conversation is every message between the same two users about the same listing, so there is no separate conversation object. The read flag is used to show how many unread messages a user has in each conversation.

Attributes:

- message_id (unique identifier)
- sender_id (references User)
- receiver_id (references User; cannot be the same as sender_id)
- listing_id (references Listing; optional, set to empty if the listing is deleted)
- body (1 to 1000 characters)
- is_read (default false)
- created_at

3.1.2 Relationships

The following relationships among data objects exist as follows:

- A User may create one Storefront.
- A Storefront may have multiple Listings.
- A Listing may have multiple Listing Images.
- A Listing may have multiple Size records (for inventory-based items).
- A User may maintain multiple Cart Items in their persistent shopping bag.
- A User (shopper) may create multiple Orders.
- An Order may contain multiple Order Items, each referencing a specific Listing.
- An Order may have zero or one associated Return record.
- A User may submit multiple Return requests.
- A User may maintain multiple Wishlist entries.
- A User may send Messages to other Users about a Listing; all Messages between the same two Users about the same Listing form one conversation. (Jonah Goodwine)
- An Administrator may access and manage all Users, Storefronts, Listings, Orders, and Returns.

These relations enforce storefront and listing ownership, transaction traceability, structured inventory management, and admin control throughout the system.

3.1.3 Complete data model

Treasure Row's Entity-Relationship Diagram (ERD) showcases the data structure of the system. The diagram shows primary entities: User, Storefront, Listing, Order, Order Item, Cart Item, Wishlist, and Return, along with their major attributes, primary keys, and foreign key relationships.

The ERD demonstrates how:

- A User may create a Storefront.
- A Storefront may hold multiple Listings.
- Listings may include associated images and size based inventory records.
- A User may maintain multiple Cart Items in their persistent shopping bag.
- Shoppers (Users) may place Orders for Listings.
- Each Order may contain multiple Order Items referencing specific Listings.
- Orders may optionally generate Return records.
- Users may save Listings to a Wishlist.
- Admins may access and manage all system entities.

The data model enforces relational integrity, ownership constraints, and transactional traceability through structured foreign key relationships. It supports authentication, listing lifecycle management, cart management, checkout, order tracking, return handling, and admin oversight within a centralized relational database environment.

3.1.4 Data dictionary

The following data dictionary defines the primary attributes used within Treasure Row system. It provides a description of each attribute for clarity of its role within the data model.

Attribute	Description	Attribute	Description
user_id	Unique identifier for each registered user	username	User created account name
email	User email address	password_hash	Securely hashed password created by user
role	Defines permission level (shopper, brand_owner, admin)	created_at	Timestamp of record creation
storefront_id	Unique identifier for a storefront	owner_id	Identifies the user who owns the storefront
contact_info	Storefront contact information	categories	Comma-delimited list of storefront style categories
preview_image_1-4	S3 URLs for storefront preview images	listing_id	Unique identifier for each listing
title	Listing title	description	Detailed listing

			information
price	Listing price	fulfillment_type	Defines whether listing is inventory based or made to order
quantity_on_hand	Current stock level	sizes_available	Available sizes for the listing
is_made_to_order	Indicates if listing is produced after purchase	status	Listing state (active, sold out, inactive, deleted)
storefront_restore_status	Remembers listing state before storefront deactivation	deleted_restore_status	Remembers listing state before soft delete
image_id	Unique identifier for listing image	image_url	Reference to image stored in Amazon S3
is_primary	Indicates primary image for listing	size_id	Unique identifier for size record
size_label	Size designation (e.g., S, M, L, XL)	quantity	Available quantity for a specific size
wishlist_id	Unique identifier for wishlist entry	order_id	Unique identifier for each order
confirmation_number	Unique system-	buyer_meeting_locatio	Location submitted

	generated order reference number	n	by buyer for in-person exchange
buyer_contact	Buyer contact information	buyer_email	Buyer email address submitted at checkout
total_amount	Total transaction amount	order_status	Order lifecycle state (SUBMITTED)
order_item_id	Unique identifier for each order line item	price_at_purchase	Listing price recorded at time of purchase
selected_size	Size selected by buyer at checkout	cart_item_id	Unique identifier for each cart item
return_id	Unique identifier for return request	order_number	Order confirmation number referenced in return
has_damage	Indicates if returned item has damage	damage_image_url	S3 URL for optional damage image
return_status	Status of return (pending, reviewed, resolved)	is_deleted	Soft delete flag for return records
message_body	Text of a message (1 to 1000 characters)	is_read	Whether the receiver has opened the conversation
sender_id / receiver_id	Users who sent and	message_listing_id	Listing a conversation

	received a message		is about (empty if deleted)

The data dictionary will be maintained electronically and updated as development progresses.

4.0 Functional Model and Description

This section describes the functional breakdown of the Treasure Row system, including processing logic, interfaces, data flow, performance considerations, and constraints. The system follows a client-server architecture deployed on AWS using Elastic Beanstalk for application hosting and Amazon RDS (PostgreSQL) for database services.

4.1 Description for Function n

Registration and Login

• The user enters name, email, and password on the Login/Sign up page
• The server (auth_services.py) checks that the email ends in @my.hamptonu.edu or @hamptonu.edu
◦ A non-Hampton email is rejected, and this is enforced not just in the browser, but on the server side
• The server checks for duplicate accounts
• The password is hashed with Werkzeug (pbkdf2: sha256), and the user record is stored with the title shopper by default
• If all the login information is valid, a Flask session is then created containing the user account and cart state, and the user is given access to the homepage
◦ If invalid information is entered, the server will return a 401 try screen
• Logout exits the session

Storefronts

• A logged-in user can click the “Create Storefront” tab, which opens /storefronts/create
• The user has to enter a store banner image & logo, brand name, contact info (instagram/ email), store description, store categories (streetwear, vintage, athleisure, accessories,

casual, earthy, retro, and other), and up to 4 preview images
◦ Each uploaded image is validated (only if it's a PNG, JPG, JPEG, GIF, or WEBP file), given a filename, uploaded to S3, and has its URL saved in the storefront table
• The new storefront appears as a card on the homepage
◦ The user will gain the benefits of a seller
◦ A user may own one storefront
•“Enter Storefront” opens /storefronts/<storefront_id>
• “Edit Storefront” runs an update on the record, and only the owner of the storefront or an admin can do so
• Inactive storefronts are hidden from the public
◦ They stay visible to the owner and admins
◦ “Deactivate Storefront” option sets is_active to false since it’s unfinished

Browse & Search

• After logging in, the shopper is directed to the storefront homepage, which loads all the active storefronts as cards (GET /api/storefronts)
• The top tabs are Storefronts, My Storefront, Sell an Item, My Bag, Returns, My Account, and About
• To the side the Menu has Shop(leads user to All Storefronts, My Bag, Returns), Sell (My Storefront, Create Storefront, Create Listing), and More(My Account, About, Sitemap, Log Out)
• Storefront cards show item count and categories with Previous-Next paging
• Create Listing allows up to 4 preview images to be showcased, size options of One Size, XS, S, M, L, XL, XXL(with quantity bar to set how much each size has) or a Made to Order option, and price
• Each card opens that storefront’s page (/storefronts/<storefront_id>), where all of its listings will be shown with the new set appearing first
◦ Listings that are deleted or hidden aren't visible to shoppers
◦ Inactive storefronts are hidden from shoppers• A listing with a quantity of 0 will show a disabled (greyed out) Sold Out instead of an add
to bag option
• ◦ Categories, although set and stored on storefronts, can't be used to filter results
◦ Listings
• The owner opens “Create Listing” and then picks one of their storefronts, enters the name of the item, adds a description, sets the price, uploads the image, and selects the size (up to 6 sizes)
◦ owner can set a quantity per size or make it made-to-order
• The server validates required fields and rejects if there are negative inputs for price/ quantity
◦ listing_sizes records this information
• Images uploaded to S3 are recorded in listing_images
• The listings show on the storefront
◦ Once the quantity of a listing reaches 0, the add to bag button is replaced by a sold

out tag
• The owner can edit listings
◦ Listing statuses can toggle between hidden and active
◦ Listings can be deleted, but it's not a definite delete
▪ They are kept in deleted_restore
• Before a listing is edited or deleted, there are checks to make sure the user owns the listing
◦ However, Admins can modify any listing

Wishlist

• A shopper can click the gold lock icon on each listing card
◦ Clicking the lock icon again can remove it
• The listing will appear in the "Wishlist" tab in My Account
◦ Displays each saved item with an add-to-cart and remove button
• The page loads the saved wishlist from the server (/api/wishlist)
◦ The browser copy is a fallback

Cart & Checkout

• The shopper selects a size and adds the item to the shopper's bag
◦ The server validates the quantity of the item on hand at the moment
◦ The price and name always come from the database
◦ A sold-out listing will display a disabled "Sold Out" label where the "Add to Cart" would be
• Items in the cart are saved to the database
◦ Loaded into the session at login
◦ Status survives logout
• On the My Bag page, the total is calculated, and the shopper can click "Checkout"
◦ The shopper enters their full name, contact info/email, and a meeting location for an in-person exchange
▪ Orders are paid with Treasure Tokens
▪ No real money or card information is collected
▪ 1 Treasure Token equals $1 of the price of the listing
◦ Stock and prices are validated again
◦ The server rejects an empty cart or a balance too low
• The server then runs these steps:
◦ Create the order (status SUBMITTED)
◦ Create the order items with price_at_purchase
◦ Move the order total from the buyer's wallet into escrow
◦ Decrement the inventory
◦ Clear the cart
◦ Remove the purchased items from the wishlist
• A confirmation number is then generated and shown on the order details page
◦ The page shows the order date, order status, order total in Treasure Tokens, buyer information, meeting location, the items purchased, and the pickup code

◦ The order can be looked up with that number
◦ Only the buyer can see the pickup code
◦ Viewing an order requires the logged-in user to be the buyer; otherwise, “Access denied” will pop up
◦ Order status:
◦ Submitted at checkout
◦ Completed once every seller’s escrow is done
◦ Cancelled when all escrows are refunded

Treasure Tokens

• Every year, a wallet opens automatically with a starting balance of 200 Treasure Tokens as a default
◦ The default amount is set by TOKEN_STARTING_BALANCE
◦ The balance is shown in the header balance, My Account, and the Wallet Tab with a purchase/sell history
• Escrow:
◦ Tokens paid at checkout are held per seller until pickup is completed
◦ Each escrow has a one-time pickup code given to the buyer (shown on the order confirmation page and Wallet tab) to give to the seller to complete the transaction
◦ The code is never stored
◦ The code is derived from the server's SECRET_KEY
• The release is at the meetup
◦ The buyer gives the seller the pickup code after they've received their items
◦ The seller enters the code on the My Account page on the Sales History tab
◦ It can also be reached from “Pickups to confirm” on My Storefront
◦ Only the seller of that escrow can release it
◦ The tokens then move to the seller’s wallet and it will show as “Picked up”
◦ After 5 wrong codes, the escrow is locked and an admin must review it
• Cancellation & Refund
◦ Before pickup, the buyer, the seller, or admin can cancel
◦ Once the refund is processed the tokens return to the buyer
◦ After pickup, the tokens can no longer be refunded by users
◦ Every balance is recorded in the ledger
◦ The wallet tab shows the history(starting balance, escrow hold, release, and refund)
◦ A balance can't go negative and checkout locks the buyer’s wallet

Access & Security

• Almost every page requires a signed-in user except for the landing page
• The user’s profile comes from the server session
• Prices always come from the database
• Forms are protected against cross-site requests
• Uploads are limited to real images under 5MB in certain folders• Hampton e-mail check rejects lookalike domains

Returns

• On the returns page, the buyer reads the Return Instructions listed at the very top then enters a confirmation number (given once an order has been confirmed), indicates whether the item is damaged or not, has an option to upload an image of the damaged item, and a return reason
• The instructions state:
• Contact the seller first
• Sellers most include contact information on their storefronts
• If the seller can't be reached then email TreasurerowAdmins@my.hamptonu.edu
• The form down below must be completed so the request can be tracked
• The server verifies the confirmation number, reason for return, that the confirmation number belongs to one of the signed-in user's own orders, and if there were answers to the damage questions
• If the confirmation number comes back invalid it will show "We couldn't find that order on your account. Check the confirmation number"
• If the item is damaged, there's an option for the buyer to upload a photo
• The upload will be checked to be a real image and if it meets the 5 MB limit
• The photo will then be stored in S3
• The return request is saved with the buyer's user_id and status pending
• An admin reviews each request in the dashboard with the buyer's username and contact information
• It will go through the process of being reviewed and then be dealt with
• The status of the request can be set to pending, reviewed, or resolved
•A return request may have one of three statuses: PENDING (submitted and awaiting review), REVIEWED (an administrator has examined the request), or RESOLVED (the review has concluded and the request has been handled). The system validates that the selected status is one of these allowed values. Submitting or resolving a return request does not automatically refund Treasure Tokens; refunds follow the separate escrow cancellation/refund rules.

• A request can jump from pending to request
• Tokens can be refunded by
• canceling before pickup
• an admin refunding an escrow that is locked
• A user submitting a return doesn't refund tokens

Admin

• The admin signs in on the Admin Access login page◦ The session store role will be admin
▪ Admin credentials come from ADMIN_USERNAME and ADMIN_PASSWORD in .env
▪ Every admin route runs require_admin(), which will read the role from the session
• The admin dashboard (/admin/dashboard) has all the users, listings, storefronts, and returns information (GET /admin/users,/listings,/storefronts,/returns)
• There are dashboard sections for wallets and escrows

•
It lists every user’s balance and grants 200 welcome tokens
• Admin can hold a list and lock pickup orders
• Admin can issue refunds to orders
• Admin can delete users, listings, and storefronts (DELETE /admin/…)
◦ When deleting a listing, the record is kept for admin tracking
▪ There’s an option to deactivate a listing; it can later be restored
• The admin updates a return’s status to pending, reviewed, or resolved (PATCH /admin/returns/<id>)
◦ The service has no priority order, so an admin can bounce from pending to resolved

Messaging (Jonah Goodwine)

• A signed-in buyer clicks "Message Seller" on a listing card in a storefront, which opens /messages?to=<seller id>&listing=<listing id>
• The Message Seller link only shows when the user is signed in and does not own that storefront
• The Messages page (/messages, also in the nav menu) shows the conversation list on the left and the open conversation on the right
• Each conversation is between two users about one listing and shows the other person's name, "Re: <listing title>", the latest message, and a red badge with the number of unread messages
• The user types a message (up to 1000 characters) and clicks Send (POST /api/messages); the server always uses the signed-in user as the sender
• The server rejects empty messages, messages over 1000 characters, messages to yourself, and unknown users or listings, and the error shows under the message box
• Opening a conversation (GET /api/messages/<other_id>) marks the messages sent to you as read and clears the badge
• The page checks for new messages every 10 seconds, so replies show up without reloading
• Message text is escaped before it is shown, so typed HTML or scripts appear as plain text
• Known limitation: messages cannot be edited or deleted, and there are no email or push notifications
• Known limitation: the admin account from .env has no user id, so it cannot use messaging (403)

4.1.1 Processing narrative (PSPEC) for function n

A processing narrative for function n is presented.

User registration:

When a user selects “Register,” the system:

- Accepts input data (name, email, password)
- Validates email format and ensures it belongs to the Hampton University domain
- Checks for duplicate accounts in PostgreSQL (RDS)
- Encrypts the password using secure hashing
- Stores user information in the Users table
- Returns a confirmation or error message

User authentication(login/logout):

- User submits email and password.
- The system retrieves the stored password hash from RDS.
- Compares the input password with the hashed value.
- If valid, it generates a secure session token.
- Grants access to marketplace features.

Create listing:

- Authenticated user selects “Create Listing.”
- User inputs:
    - Item name
    - Description
    - Price
    - Category
    - Image upload
- System validates required fields
- Stores listing in Listings table in RDS
- Uploads image to AWS storage (S3). Displays listing in the marketplace feed

Edit/delete listing:

- User selects the listing they own
- System verifies ownership
- Allows modification or deletion
- Updates or removes listing from RDS

Browse & search listings

Current behavior: Search is limited to storefront names. Listing-level keyword search, price filtering, category filtering, and sorting are not currently available. If these functions are required, they should be listed as planned requirements with separate acceptance tests; otherwise, remove the unsupported claim that the current search applies those filters.

Admin management:

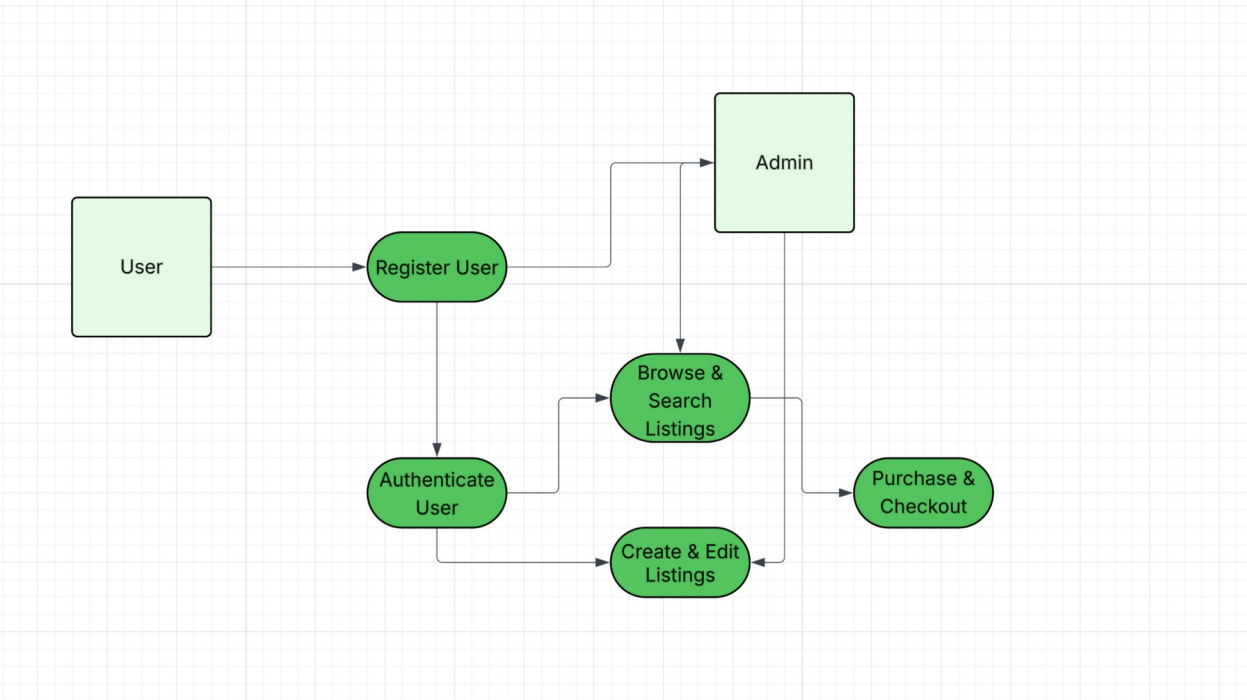
Admin users can:

- View all listings
- Remove inappropriate content
- Suspend user accounts

Purchase & checkout:

- User selects item
- Confirms purchase
- System verifies listing availability
- Marks item as sold in the database
- Notifies seller

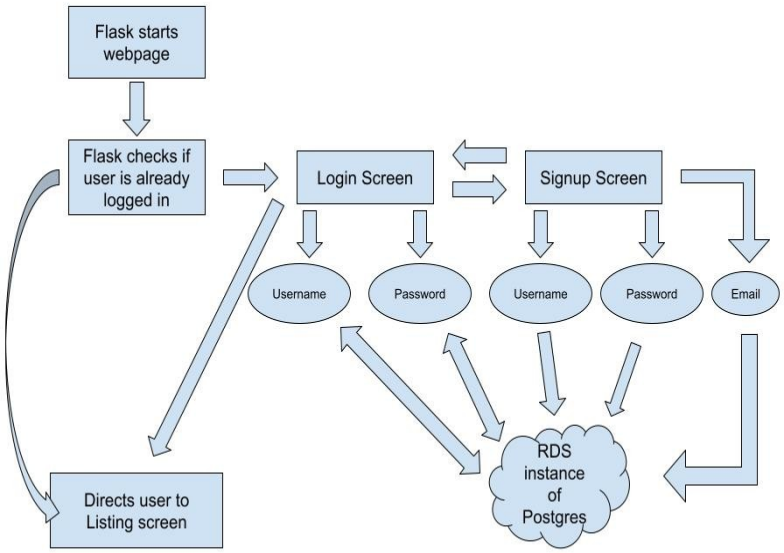
4.1.2 Function n flow diagram



4.1.3 Function n interface description

A detailed description of the input and output interfaces for the function is presented.

- Input interface: JSON is a list of keys and values. During, the input interface expects three pieces of input: email, username, and password. If the data type doesn't match the JSON rules, such as a phone number being inputted instead of an email, the interface rejects it at the window before it reaches the database.
- Output interface: If the login is successful, a 200 code response is sent back along with routing to the listing screen. But, if the login fails, a 401 error will be sent.



4.1.4 Function n transforms

A detailed description for each transform (subfunction) for function n is presented. Section 4.1.4 is repeated for each of k transforms.

4.1.4.1 Transform 1 description (processing narrative, PSPEC)

This transform manages how students join and enter the marketplace.

During registration, the system accepts a name, email, and password. During this process, the system ensures the email belongs to the Hampton University domain (@hamptonu.edu), the username/email isn't a duplicate, and the password is hashed before storing it in the PostgreSQL database on AWS RDS.

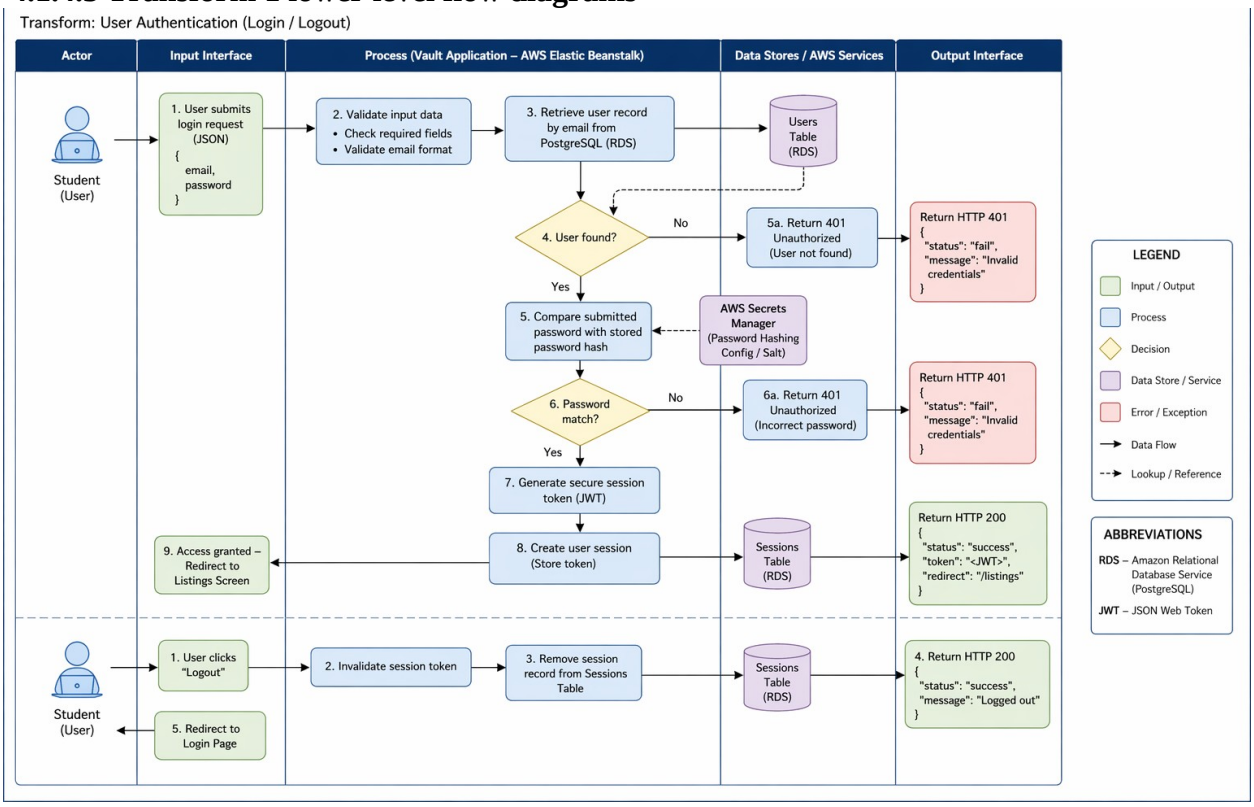
When logging in, the system retrieves the stored hash and compares it to the input password. If they match, a secure session token is generated to grant access to the site's features.

4.1.4.2 Transform 1 interface description

For the input interface, the system expects a JSON list containing the user's email, username, and password. If the data types are incorrect, the interface rejects the input before it even reaches the database.

For the output interface, a "200" success code is returned during a successful login and routes the user to the listing screen. A failed login sends a "401" error code.

4.1.4.3 Transform 1 lower level flow diagrams



4.1.4.4 Transform 2 description

This transform handles how brand owners control their products.

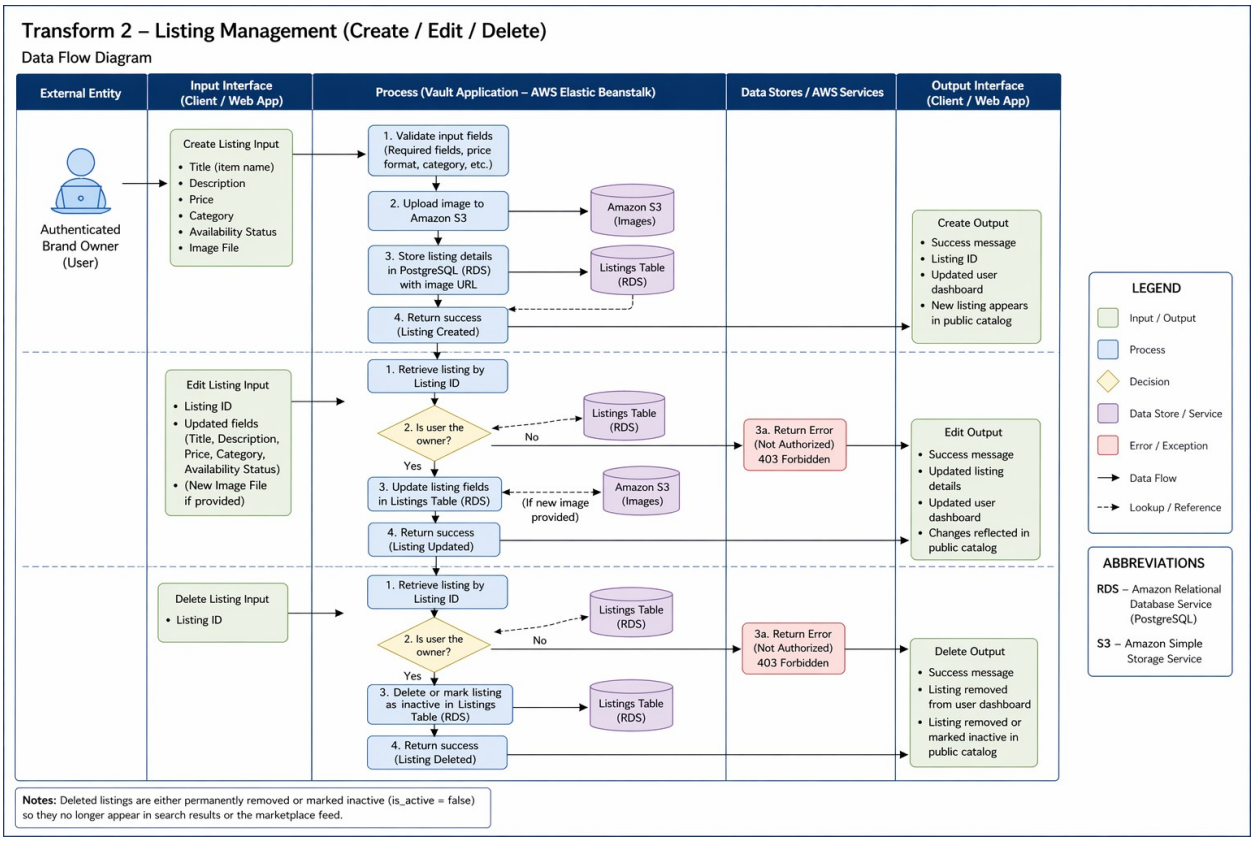
During listing creation, authenticated users input an item's name, description, price, and category. The system validates the fields, stores the details in the RDS database, and uploads the product image to Amazon S3 storage.

When it comes to edits and deletion, the system verifies that the user is the actual owner of the listing before allowing any modifications or deletions. If deleted, the record is removed or marked inactive so it doesn't appear in the feed anymore.

4.1.4.5 Transform 2 interface description

For the input interface, item details (title, price, availability status) are provided during listing creation. For the output interface, a public catalog of active listings and updated user dashboards showing their managed storefront is displayed.

4.1.4.6 Transform 2 lower-level flow diagrams



4.1.4.7 Transform 3 description

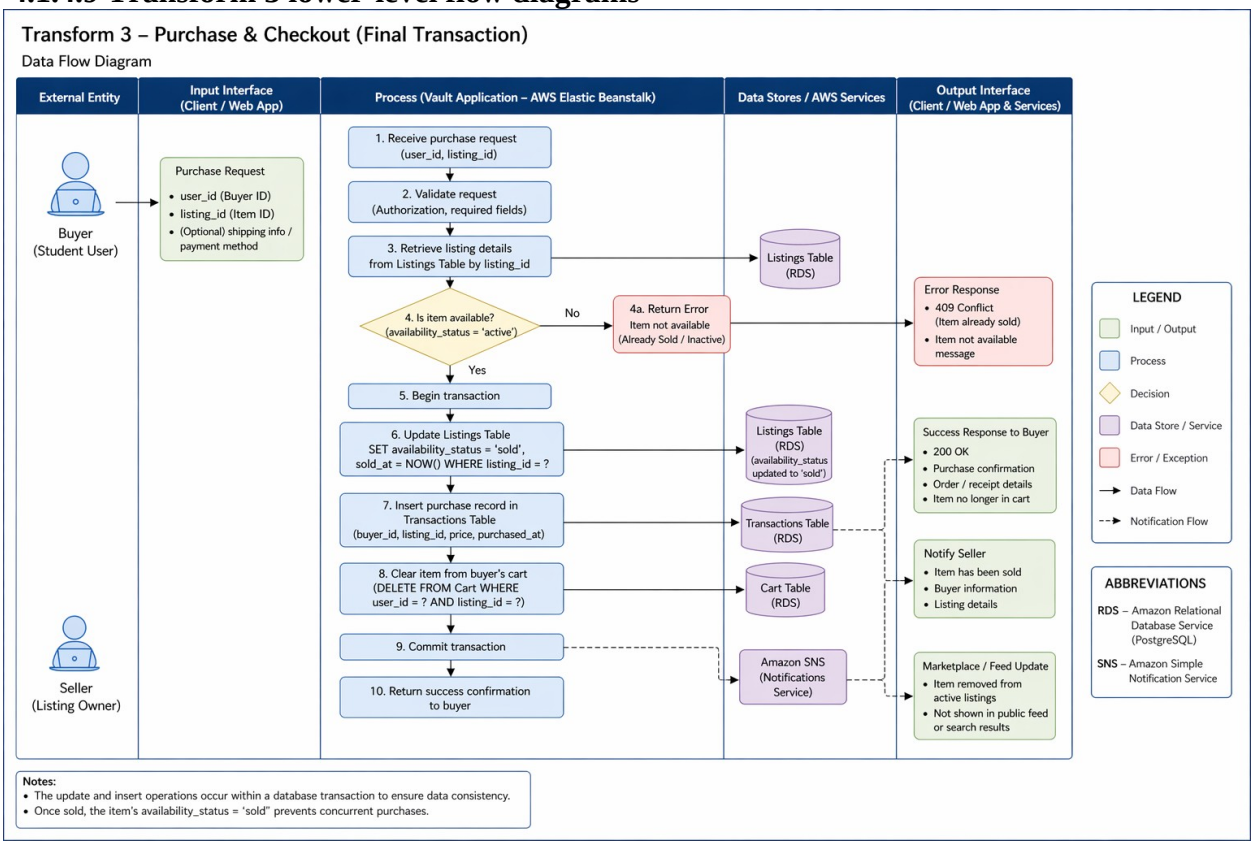
This transform handles the final transaction phase where a student buyer confirms their interest and purchases the item from the marketplace.

At checkout, the server must revalidate cart contents, current prices, inventory, and the buyer's Treasure Token balance. If validation succeeds, it creates an order and its order items, records each item's purchase-time price, places the applicable token amount into escrow, updates inventory, clears the cart, and removes purchased listings from the buyer's wishlist. The order remains submitted until the documented pickup, cancellation, and escrow rules determine its final status. If any required step fails, the transaction must not leave a partially completed order or inconsistent balance/inventory.

4.1.4.8 Transform 3 interface description

The inputs are the User_ID of the buyer and the Listing_ID of the specific item being purchased, and the outputs are the updated “Sold” status in the database, a confirmation message to the buyer, and a notification sent to the seller. In addition, the item is removed from other users’ feeds.

4.1.4.9 Transform 3 lower-level flow diagrams



4.1.5 Performance Issues

Special performance requirements for the subsystem are specified.

- Performance target: Under the agreed test dataset and load, 95% of database queries in the defined test set should complete within 200 ms, measured at the database layer. Record the test dataset, concurrency level, and measurement method. For checkout, prioritize correctness under concurrent purchases: the system must prevent overselling and inconsistent token balances, using appropriate database transactions and concurrency controls.
- Concurrency: The system should be able to handle multiple simultaneous user inputs without locking database rows.

4.1.6 Design Constraints

Any design constraints that will impact the subsystem are noted.

- Cloud security: The system is constrained by the AWS Security Group Inbound Rules. These only accept traffic from authorized IP addresses.
- Statelessness: Sessions are stored in secure cookies instead of files to allow scalability in Flask.

4.2 Software Interface Description

The software interface(s)to the outside world is(are) described.

4.2.1 External machine interfaces

Interfaces to other machines (computers or devices) are described.

- Psycopg2 communicates with AWS RDS over port 5432. This connection is initialized in init_db.py and tested in db.py.

4.2.2 External system interfaces

Interfaces to other systems, products, or networks are described.

- Treasure Row stores structured application data in an Amazon RDS for PostgreSQL database hosted in the project's configured AWS Region. The team's deployment is constrained by the AWS account's available service quotas, free-tier eligibility, and budget. Database schema changes are applied through the project's initialization or migration scripts and an authorized database connection; the team should document the actual workflow used.

- Internet Browser: The server interfaces with the browser using JSON for data and HTML/CSS for visuals.
- **AWS Elastic Beanstalk and network access (Jonah Goodwine):** The app is deployed to the Elastic Beanstalk environment treasurerow-env (us-east-2) and stores images in S3. The RDS security group allows PostgreSQL on port 5432 only from the Beanstalk server, approved team member IP addresses, and the Hampton campus range; it never allows 0.0.0.0/0. Access, troubleshooting, redeploy and shutdown steps are in docs/aws-runbook/README.md.

4.2.3 Human interface

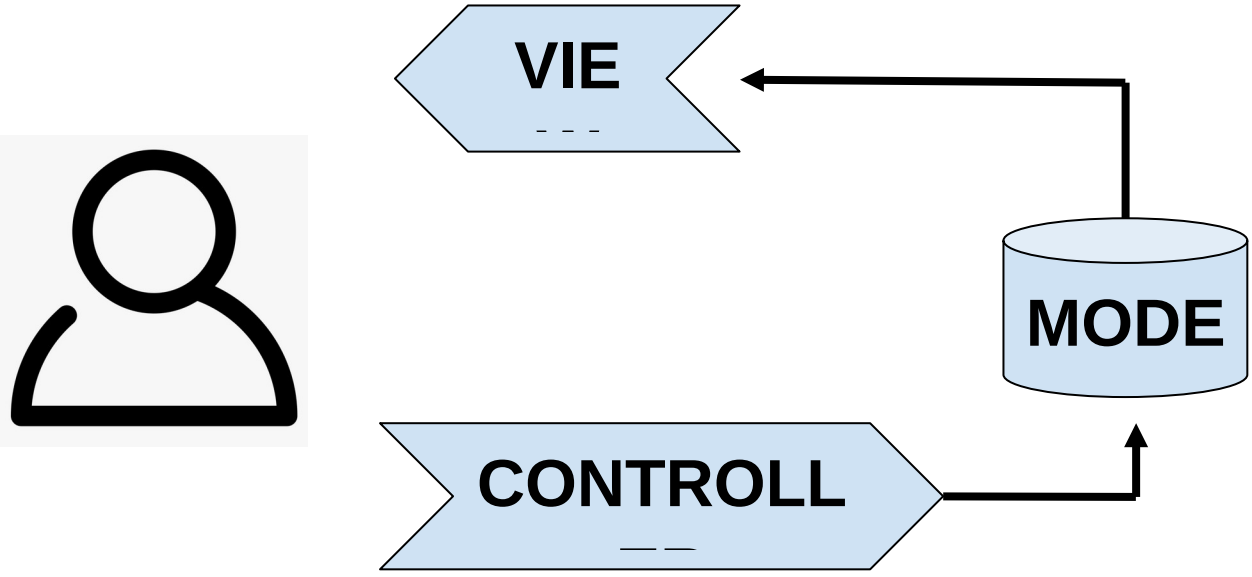
An overview of any human interfaces to be designed for the software is presented.

- Web GUI: A webpage at https://127.0.0.1:5000 will contain the webpage frontend.
- Admin CLI: init_db.py allows for direct database schema management.
- **Live site (Jonah Goodwine):** http://treasurerow.us-east-2.elasticbeanstalk.com. Local development runs at http://127.0.0.1:5001.

4.3 Control flow description

The control flow for the system is presented with reference to Section 5.0 of this document.

1. Entry: Requests go to app.py.
2. Routing: app.py delegates to a routing blueprint.
3. Logic: The controllers validate the formatting of the requests and hand them to the services.
4. Data: The services request data from the models; the models execute the SQL commands on the AWS RDS.
5. Exit: Data flows back up the chain and is returned to the user as JSON.



5.0 Behavioral Model and Description

A description of the behavior of the software is presented.

5.1 Description for software behavior

This section describes the dynamic behavior of Treasure Row system, including the major system states and the events that trigger transitions between those states. The behavioral model illustrates how the system responds to user interactions, administrative actions, and internal validation processes.

The application uses Flask session-based authentication and role-based authorization. Requests that require authentication rely on the session to identify the signed-in user. Describe the session storage mechanism actually used by the application (for example, a signed client-side cookie or a server-side session store) and state separately whether API endpoints are designed to be stateless between requests.

5.1.1 Events

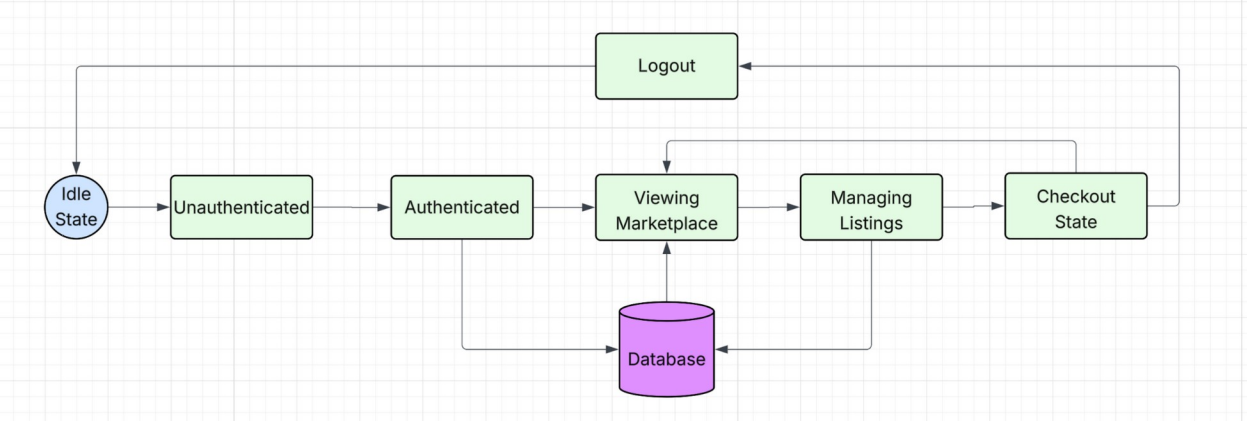
Event	Description
Register Submitted	User provides information to register
Login Submitted	User tries logging in
Create listing clicked	User begins creating their listing
Submit listing	User submits their listing
Edit listing clicked	User edits their listing
Delete listing clicked	User removes their listing
Search performed	User searches for specific listings
Purchase confirmed	User confirms purchase
Admin remove	Admin removes a listing
Logout	User ends session and leaves the webpage
Send message (Jonah Goodwine)	User sends a message about a listing; it is saved as unread for the receiver
Open conversation (Jonah Goodwine)	Messages sent to the user in that conversation are marked read

5.1.2 States

The system operates under the following primary states:

1. Unauthenticated State
2. Authenticated State
3. Viewing Marketplace
4. Creating Listing
5. Editing Listing
6. Checkout State
7. Admin State

5.2 State Transition Diagrams



5.3 Control specification (CSPEC)

The system uses event-driven control logic:

- User input triggers events.
- Backend validates state.
- If valid, state transition occurs.
- If invalid, error state is returned.

Security control rules:

- Only Hampton University students can use *Treasure Row*, security verification th
- Only authenticated users may create/edit/delete listings.
- Only admin users may remove content.
- Session timeout returns system to Unauthenticated state.

6.0 Restrictions, Limitations, and Constraints

Restrictions

- **Hampton University-Only Access:** Treasure Row is restricted to use by Hampton University students only. A Hampton University email address is required to sign in

and/or create a new account. The system will validate if it is a Hampton University email account based on the ending of the email address being (hamptonu.edu).

- **Role-Based Access Control:** Users are restricted based on assigned roles. Brand owners and shoppers cannot access administrative controls and only administrators can manage all users and listings. This restriction enforces security and ownership boundaries in the system.
- **AWS Free Tier Deployment Requirement:** The system must be deployed using AWS Free Tier services. This restricted available computing resources, database sizes and storage limits, and performance abilities.

Limitations

- **AWS RDS PostgreSQL Query Editing Limitation:** When Postgres is implemented through RDS AWS the free tier does not allow for the query to be edited directly through AWS. To circumvent this, we made an init_db.py in VS Code that can initialize the tables in Postgres without having to upgrade the AWS plan
- **Scalability Limitation Under Free Tier:** Because the project has to be operated within the AWS Free Tier, the database storage, memory allocation, and connections are limited. Performance under high traffic or large numbers of users may be limited.
- **Development Timeline Limitation:** The project is limited by the semester timeline. Some advanced features may not be fully implemented before the project is completed.

Constraints

- **Technical Constraints:** The system must be compatible with AWS Hosting services, specifically AWS RDS for database management. The application must support cross-platform browser compatibility/access like macOS and Windows. PostgreSQL is the required relational database system for efficient storage.
- **Security Constraints:** Passwords must be stored using secure hashing methods. Sensitive payment information cannot be stored in the database system. Listing ownership must be enforced at database and application levels.
- **Resource Constraints:** The team must work within the AWS Free tier system. Development must also be completed within the project iteration timelines.

7.0 Validation Criteria

Login Page

- Users are able to create an account
- Users are able to login to their already existing account
- The account login information will be stored into the database
- Once the login process is complete, the user will be brought to the listing page

Listings Page

- Users can view several different listing posted
- Users can create their own listing
- Users can edit their existing listing
- Users can select to buy a listing
- All user-created listing information will be stored into the database
- After the user selects to buy a listing, they will be brought to the checkout section

Checkout Page

- Users can confirm their purchase and submit their payment

7.1 Classes of tests

The types of tests to be conducted are specified, including as much detail as is possible at this stage. Emphasis here is on black- box testing.

- **Unit Testing:** This is when a tester tests small chunks of code, like a single function or method, to make sure it does what it's supposed to. The goal is to confirm that the logic the developer wrote works the way they intended (GeeksforGeeks, 2025). This will be easily implemented by Treasure Row because programmers will test functionality before changes are committed to the shared repository.
- **Integration Testing:** This type of testing checks how different pieces of the software work together. It makes sure that data flows correctly between components and that nothing breaks when modules are combined (GeeksforGeeks, 2025). This will consistently be implemented individually and collaboratively by validating changes before commitment and validating changes on each individual's desktop after commitment.
- **System Testing:** Once everything is integrated, system testing looks at the entire application as a whole. It checks if the full system meets the requirements and works the way it's supposed to (GeeksforGeeks, 2025). This won't be as heavily desired since a finished, working product isn't expected by the customer.
- **Black-Box Testing:** In black-box testing, the tester doesn't need to know how the system is built on the inside. They just provide inputs and check the outputs. This helps reveal how the system reacts to different user actions, whether expected or not, and can point out issues with usability, reliability, or performance. Common types include functional, non-functional, and regression testing (GeeksforGeeks, 2025;Imperva, n.d.). This may be difficult to implement because everyone is a programmer, so we're expected to know how the system is built on the inside. To mitigate this issue, we can have individuals who did not code the tested functions test the functions.
- **White-Box Testing:** In white-box testing the tester has full access to the code, structure, and configuration. It checks the logic, data flow, and error handling to make sure everything inside the program runs the way it should (GeeksforGeeks, 2025; Imperva, 2025). This will be heavily and consistently used throughout coding. White-box testing should be done by every member of the team, and the software lead should do a very in-depth white-box testing.

7.2 Expected software response

- **Scalability:** Ability to process high volumes of listings and users.
- **Security Compliance:** No unauthorized access to restricted data or features, encrypted storage of sensitive data.

- **Usability:** Clear and accessible interface, ensuring seamless user interactions.
- **Fault Tolerance:** The system must be able to remain stable during unexpected failures, with automatic recovery and fallback mechanisms.

7.3 Performance bounds

- **Admin Security:** Only authorized users should be able to access that admin page.

8.0 Revision Logs

Revision	By	Date	Description
1.0	Treasure Row	1/13/26-1/22/26	The initial document was created, and an outline and table of contents were added.
2.0	Treasure Row	1/23/26-2/12/26	Updated majority of the previously unfilled selections (2, 3, & 7)
3.0	Treasure Row	2/12/2026- 3/5/26	Updated Section 6 based on additional restrictions for the project, updated sections 4-5 with additional diagrams and functions present in our code, added a section 4.4 to include each team member's code contributions and descriptions
4.0	Treasure Row	3/5/26-3/26/26	Added code contributions for iteration 4

5.0	Treasure Row	4/9/26 - 4/23/2026	Added I5 code contributions for all team members (Sections 4.4.1-4.4.6). Updated data model sections 3.1.1.7-3.1.1.10 to reflect actual Order, Order Item, Cart Item, and Return objects. Updated 3.1.2 relationships, 3.1.3 complete data model, and 3.1.4 data dictionary to remove payment references and add cart, order, and return entities. Updated 3.0 and 3.1 intro to reflect completed checkout and order tracking system.

6.0	Noah Higgs	10/27/2026	Registration now enforces a username of 3 to 50 characters and a password of 8 to 128 characters; failed logins close their database connection; every response carries security headers; the app refuses a placeholder SECRET_KEY; order confirmation numbers use the secrets module.

7.0	Jonah Goodwine	10/27/2026	Added the Message data object (3.1.1.11), messaging relationship and data dictionary entries, the Messaging walkthrough in 4.1, messaging events in 5.1.1, and the Elastic Beanstalk deployment and live site (4.2.2, 4.2.3). Removed the inherited team’s code contribution sections (former 4.4) and replaced the old project name with Treasure Row.